

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

**Đề tài: Finetune mô hình dịch máy Neural cho
văn bản chuyên ngành y khoa Anh-Việt**

Lớp tín chỉ:	D23CQCN04-B
Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Đức Trung
Mã sinh viên:	B23DCCN858

Hà Nội 2026

MỤC LỤC

1. GIỚI THIỆU	5
1.1 Bối cảnh và vấn đề	5
1.2 Mục tiêu dự án	5
1.3 Phạm vi và giới hạn	5
2. YÊU CẦU NGHIỆP VỤ	6
2.1 Yêu cầu chức năng	6
2.2 Yêu cầu phi chức năng	6
3. KIẾN TRÚC HỆ THỐNG TỔNG THỂ	7
3.1 Tổng quan kiến trúc	7
3.2 Luồng dữ liệu end-to-end	7
3.3 Kiến trúc phân lớp Backend	7
4. CÔNG NGHỆ	8
4.1 Bảng tổng hợp công nghệ	8
4.2 Lý do lựa chọn công nghệ	8
5. THIẾT KẾ FRONTEND	10
5.1 Cấu trúc thư mục dự án	10
5.2 Các thành phần giao diện chính	10
5.3 Quản lý trạng thái với Zustand	10
5.4 Axios Interceptor	11
5.5 Tính năng UX nổi bật	11
6. THIẾT KẾ BACKEND	12
6.1 Kiến trúc phân tầng	12
6.2 Module Authentication	12
6.3 Module Conversation & Message	12
6.4 Bảo mật JWT và Spring Security	13
6.5 Xử lý lỗi toàn cục	13
6.6 Tích hợp AI qua WebClient	13
7. THIẾT KẾ CƠ SỞ DỮ LIỆU	15
7.1 Chi tiết bảng users	15
7.2 Chi tiết bảng conversations	15
7.3 Chi tiết bảng messages	15
8. TÍCH HỢP AI INFERENCE	17
8.1 Mô hình sử dụng	17

8.2 Giao thức giao tiếp với FastAPI	17
8.3 Xử lý hướng dịch	17
8.4 Đo lường Latency	17
8.5 Xử lý lỗi AI Service.....	18
9. CÁC CHỨC NĂNG CHÍNH VÀ LƯỒNG HOẠT ĐỘNG.....	19
9.1 Đăng ký và Đăng nhập	19
9.2 Quản lý Conversation	19
9.3 Dịch thuật thời gian thực.....	19
9.4 Danh sách API Endpoints.....	19
10. BẢO MẬT	20
10.1 Xác thực và Phân quyền.....	20
10.2 Bảo mật mật khẩu và JWT	20
10.3 Validation dữ liệu đầu vào.....	20
10.4 Cấu hình CORS	20
11. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY DỰ ÁN	21
11.1 Yêu cầu hệ thống.....	21
11.2 Bước 1: Khởi tạo Cơ sở dữ liệu	21
11.3 Bước 2: Cấu hình Backend	21
11.4 Bước 3: Chạy Backend.....	22
11.5 Bước 4: Cấu hình và chạy Frontend	22
12. KẾT QUẢ ĐẠT ĐƯỢC	23
12.1 Danh sách tính năng hoàn thành	23
12.2 Hiệu năng hệ thống.....	23
12.3 Mô tả giao diện người dùng	23
13. HƯỚNG PHÁT TRIỂN	24
13.1 Tính năng nâng cao.....	24
14. KẾT LUẬN	24

TÓM TẮT DỰ ÁN

MedTrans là hệ thống dịch thuật y khoa trực tuyến được thiết kế nhằm hỗ trợ dịch nhanh và chính xác nội dung chuyên môn giữa tiếng Anh và tiếng Việt. Hệ thống kết hợp giao diện web hiện đại xây dựng bằng React, backend RESTful với Spring Boot, cơ sở dữ liệu MySQL và dịch vụ AI Inference thông qua FastAPI sử dụng mô hình ngôn ngữ EnViT5.

Các điểm nổi bật của giải pháp bao gồm:

- Xác thực người dùng bảo mật thông qua cơ chế JWT stateless.
- Quản lý hội thoại và lịch sử tin nhắn theo từng phiên dịch.
- Tích hợp AI Inference để thực hiện dịch thuật y khoa.
- Hiển thị kết quả dịch kèm chỉ số độ trễ (latency_ms) theo thời gian thực.
- Kiến trúc phân lớp rõ ràng, tách biệt frontend/backend, tuân thủ các best practices bảo mật cơ bản.
- Giao diện người dùng đơn giản, trực quan, hỗ trợ dark mode và responsive layout.

Dự án được xây dựng với mục tiêu chứng minh khả năng thiết kế và triển khai một hệ thống phần mềm hoàn chỉnh trong lĩnh vực y tế, phù hợp để mở rộng thành sản phẩm thực tế trong tương lai.

1. GIỚI THIỆU

1.1 Bối cảnh và vấn đề

Trong lĩnh vực y tế, việc dịch thuật chính xác giữa tiếng Anh và tiếng Việt đóng vai trò then chốt. Tài liệu y khoa, hồ sơ bệnh án, hướng dẫn điều trị và giao tiếp chuyên môn đòi hỏi độ chính xác cao, trong khi tốc độ xử lý cũng là yếu tố quan trọng trong môi trường lâm sàng. Tuy nhiên, thực tế hiện nay cho thấy một số hạn chế đáng kể:

- Chuyên gia dịch thuật y khoa không phải lúc nào cũng sẵn sàng, đặc biệt trong các tình huống khẩn cấp.
- Các công cụ dịch phổ thông không được tối ưu hóa cho ngôn ngữ y tế chuyên ngành, dẫn đến nguy cơ sai lệch nghĩa.
- Thiếu hệ thống quản lý lịch sử hội thoại và phiên dịch để theo dõi, tham chiếu lại nội dung.

1.2 Mục tiêu dự án

Dự án MedTrans hướng đến các mục tiêu cụ thể sau:

1. Xây dựng ứng dụng web tương tác cho phép dịch thuật hai chiều Anh–Việt và Việt–Anh.
2. Cung cấp hệ thống đăng ký và đăng nhập bảo mật thông qua JWT.
3. Quản lý toàn diện vòng đời conversation và lịch sử tin nhắn.
4. Tích hợp mô hình AI EnViT5 thông qua FastAPI để xử lý dịch thuật y khoa.
5. Hiển thị độ trễ dịch và cung cấp tính năng sao chép kết quả tiện lợi.

1.3 Phạm vi và giới hạn

Phạm vi hiện tại của hệ thống bao gồm:

- Dịch thuật văn bản y khoa (text-based) theo hai hướng EN→VI và VI→EN.
- Quản lý người dùng, conversation và message với phân quyền cơ bản.
- Gọi AI Inference qua FastAPI /predict và đo lường latency.

Giới hạn hiện tại của hệ thống:

- FastAPI là dịch vụ bên ngoài, độ ổn định phụ thuộc vào host triển khai.
- Chưa tích hợp rate limiting hoặc các cơ chế chống tấn công nâng cao.
- Kiểm soát truy cập theo vai trò hiện chỉ ở mức cơ bản.

2. YÊU CẦU NGHIỆP VỤ

2.1 Yêu cầu chức năng

STT	Chức năng	Mô tả
1	Đăng ký tài khoản	Người dùng đăng ký bằng email, username và password.
2	Đăng nhập	Đăng nhập và nhận JWT token để xác thực.
3	Tạo conversation	Khởi tạo phiên dịch thuật mới.
4	Liệt kê conversations	Xem danh sách các phiên dịch hiện có.
5	Xóa conversation	Xóa phiên dịch và toàn bộ tin nhắn liên quan.
6	Xem lịch sử message	Xem lại toàn bộ nội dung đã dịch trong một phiên.
7	Dịch văn bản y khoa	Gửi yêu cầu dịch và nhận kết quả từ AI.
8	Hiển thị latency	Hiển thị thời gian phản hồi AI cho mỗi tin nhắn.
9	Sao chép kết quả	Sao chép nội dung dịch vào clipboard.
10	Chọn hướng dịch	Chuyển đổi giữa EN→VI và VI→EN.

2.2 Yêu cầu phi chức năng

Tiêu chí	Yêu cầu
Hiệu năng	Backend phản hồi trong vài trăm ms; latency tổng phụ thuộc mô hình AI.
Bảo mật	JWT stateless, BCrypt password hashing, kiểm soát route qua Spring Security.
Khả dụng	Giao diện trực quan, hỗ trợ dark mode, auto-resize textarea, phím tắt Enter để gửi.
Bảo trì	Kiến trúc phân lớp rõ ràng, DTO tách biệt Entity, code dễ mở rộng.
Khả chuyển	Frontend chạy trên Vite; Backend Spring Boot cấu hình qua environment variables.

3. KIẾN TRÚC HỆ THỐNG TỔNG THỂ

3.1 Tổng quan kiến trúc

Hệ thống MedTrans được xây dựng theo mô hình kiến trúc phân lớp (Layered Architecture), tách biệt rõ ràng các thành phần Frontend, Backend, Database và AI Service. Mô hình này đảm bảo tính module hóa, dễ bảo trì và khả năng mở rộng trong tương lai.

3.2 Luồng dữ liệu end-to-end

1. Người dùng truy cập giao diện web qua trình duyệt.
2. Sau khi đăng nhập, frontend lưu JWT token và đính kèm vào mọi request qua Axios interceptor.
3. Người dùng nhập nội dung cần dịch, chọn hướng dịch EN_VI hoặc VI_EN.
4. Frontend gửi POST /api/messages/translate kèm JWT token.
5. Backend xác thực JWT, kiểm tra quyền truy cập và conversation ownership.
6. Backend lưu message người dùng vào bảng messages.
7. Backend gọi FastAPI /predict qua WebClient với prompt được chuẩn hóa.
8. Nhận phản hồi AI, tính latency, lưu message AI và trả kết quả về frontend.
9. Frontend hiển thị kết quả dịch, latency và cung cấp nút sao chép.

3.3 Kiến trúc phân lớp Backend

Backend Spring Boot được tổ chức theo 4 tầng chức năng rõ ràng:

- Controller Layer: Tiếp nhận HTTP request, validate đầu vào, trả về HTTP response.
- Service Layer: Xử lý toàn bộ logic nghiệp vụ, điều phối các thành phần.
- Repository Layer: Tương tác với cơ sở dữ liệu thông qua Spring Data JPA.
- Entity / DTO Layer: Định nghĩa mô hình dữ liệu và đối tượng truyền dữ liệu.

4. CÔNG NGHỆ

4.1 Bảng tổng hợp công nghệ

Phân lớp	Công nghệ	Phiên bản	Vai trò
Frontend	React	18	Xây dựng giao diện SPA
	Vite	Latest	Build tool & dev server
	TailwindCSS	3.x	Utility-first CSS, dark mode
	Zustand	4.x	State management nhẹ, persist
	Axios	1.x	HTTP client, JWT interceptor
	React Router	v6	Điều hướng SPA
	React Markdown	Latest	Render nội dung Markdown
	React Hot Toast	Latest	Thông báo toast
Backend	Java	17	Ngôn ngữ lập trình chính
	Spring Boot	3.x	REST API, business logic
	Spring Security	6.x	Authentication & Authorization
	Spring Data JPA	3.x	ORM tương tác MySQL
	Spring WebFlux	6.x	WebClient gọi AI service
	Lombok	Latest	Giảm boilerplate code
Database	springdoc-openapi	2.x	Swagger/OpenAPI documentation
	MySQL	8.x	Lưu trữ user, conversation, message
AI Service	FastAPI	Latest	AI Inference server
	EnViT5	Latest	Mô hình dịch thuật EN↔VI
Build	Maven	3.x	Build & dependency management Backend
	npm / Vite	Latest	Build & package management Frontend

4.2 Lý do lựa chọn công nghệ

Các công nghệ được lựa chọn dựa trên các tiêu chí về hiệu quả phát triển, độ phổ biến trong cộng đồng và tính phù hợp với yêu cầu dự án:

- React + Vite: Tốc độ phát triển nhanh, Hot Module Replacement (HMR) giúp trải nghiệm phát triển tốt.

- TailwindCSS: Dễ dàng xây dựng UI responsive, tích hợp dark mode thông qua class strategy.
- Zustand: State management tối giản, hỗ trợ persist middleware để lưu token và UI settings.
- Spring Boot: Nền tảng Java enterprise mạnh mẽ, dễ cấu hình, hệ sinh thái phong phú.
- Spring Security + JWT: Cơ chế xác thực stateless hiện đại, phù hợp với SPA.
- WebClient: Hỗ trợ gọi API bất đồng bộ, cấu hình retry và timeout linh hoạt.
- MySQL: Phổ biến, dễ triển khai, hỗ trợ đầy đủ các tính năng quan hệ cần thiết.
- FastAPI: Framework Python nhẹ, hiệu năng cao, phù hợp cho AI/ML serving.

5. THIẾT KẾ FRONTEND

5.1 Cấu trúc thư mục dự án

```
frontend/  
src/  
  api/  
    client.js      # Axios instance & interceptors  
  components/  
    ChatArea.jsx   # Vùng hiển thị & gửi tin nhắn  
    Header.jsx     # Chọn direction, dark/light mode  
    Sidebar.jsx    # Danh sách conversation  
  pages/  
    Chat.jsx       # Layout chính của ứng dụng  
    Login.jsx      # Trang đăng nhập  
    Register.jsx   # Trang đăng ký  
  store/  
    auth.js        # Zustand auth store  
    ui.js          # Zustand UI store  
  App.jsx          # Định nghĩa routes  
  main.jsx         # Entry point  
  index.css        # TailwindCSS directives
```

5.2 Các thành phần giao diện chính

Ứng dụng được tổ chức thành các component chức năng rõ ràng:

- Chat.jsx: Layout container chính, quản lý danh sách conversation và conversation đang hoạt động. Kết hợp Sidebar, Header và ChatArea.
- Sidebar.jsx: Hiển thị danh sách conversation của người dùng, nút tạo mới conversation, nút đăng xuất. Responsive, có thể ẩn/hiện trên mobile.
- Header.jsx: Thanh điều hướng trên cùng, cho phép chọn hướng dịch (EN→VI / VI→EN) và chuyển đổi dark/light mode.
- ChatArea.jsx: Khu vực chat trung tâm, tải lịch sử tin nhắn khi chọn conversation, gửi yêu cầu dịch, hiển thị bubble tin nhắn với latency và nút copy.
- Login.jsx / Register.jsx: Form xác thực đơn giản với validation và thông báo lỗi.

5.3 Quản lý trạng thái với Zustand

Ứng dụng sử dụng hai Zustand store được persist vào localStorage:

- useAuthStore (medtrans-auth): Lưu trữ JWT token và thông tin người dùng đăng nhập.
- useUiStore (medtrans-ui): Lưu trữ trạng thái dark mode và hướng dịch đang được chọn.

5.4 Axios Interceptor

Cơ chế interceptor được cấu hình để xử lý xác thực tự động:

```
// Request interceptor - Tự động thêm JWT header
api.interceptors.request.use((cfg) => {
  const token = useAuthStore.getState().token;
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  return cfg;
});

// Response interceptor - Tự động logout khi nhận 401
api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) useAuthStore.getState().logout();
    return Promise.reject(err);
  }
);
```

5.5 Tính năng UX nổi bật

- Auto-resize textarea: Textarea tự động điều chỉnh chiều cao theo nội dung nhập.
- Keyboard shortcut: Enter để gửi tin nhắn; Shift+Enter để xuống dòng.
- Latency indicator: Mỗi tin nhắn AI hiển thị biểu tượng ⚡ kèm thời gian phản hồi (ms).
- Copy button: Nút sao chép kết quả dịch vào clipboard với thông báo xác nhận.
- Dark mode: Toàn bộ UI hỗ trợ dark mode thông qua TailwindCSS darkMode: class.
- React Markdown: Nội dung phản hồi AI được render dưới dạng Markdown để trình bày đẹp hơn.

6. THIẾT KẾ BACKEND

6.1 Kiến trúc phân tầng

Backend Spring Boot được tổ chức theo cấu trúc phân lớp chuẩn với các package sau:

```
backend/src/main/java/com/medtrans/  
  controller/  
    AuthController.java    # Xử lý đăng ký, đăng nhập  
    ChatController.java    # Xử lý conversation, message, dịch thuật  
  service/  
    AuthService.java       # Logic xác thực người dùng  
    ChatService.java       # Logic hội thoại và dịch thuật  
    AiClient.java          # Giao tiếp với FastAPI service  
  repository/  
    UserRepository.java  
    ConversationRepository.java  
    MessageRepository.java  
  entity/  
    User.java  
    Conversation.java  
    Message.java  
  dto/  
    Dtos.java              # Tổng hợp tất cả DTO  
  security/  
    JwtUtils.java          # Generate & validate JWT  
    JwtAuthFilter.java     # Filter xác thực token  
  config/  
    SecurityConfig.java    # Spring Security configuration  
    WebClientConfig.java   # WebClient configuration  
  exception/  
    ApiException.java      # Custom exception  
    GlobalExceptionHandler.java # Xử lý lỗi toàn cục
```

6.2 Module Authentication

AuthController cung cấp hai endpoint công khai không yêu cầu xác thực:

- POST /api/auth/register: Kiểm tra email/username tồn tại, mã hóa password bằng BCrypt, tạo user, generate JWT.
- POST /api/auth/login: Xác thực credential, generate và trả JWT token.

6.3 Module Conversation & Message

ChatController quản lý toàn bộ vòng đời của conversation:

- GET /api/conversations: Lấy danh sách conversation của user hiện tại, sắp xếp theo createdAt giảm dần.
- POST /api/conversations: Tạo conversation mới với tiêu đề mặc định "New chat".
- DELETE /api/conversations/{id}: Kiểm tra ownership trước khi xóa; trả 403 nếu không có quyền.
- GET /api/conversations/{id}/messages: Lấy danh sách message theo thứ tự createdAt tăng dần.
- POST /api/messages/translate: Xử lý yêu cầu dịch thuật đầy đủ (lưu message, gọi AI, lưu kết quả).

6.4 Bảo mật JWT và Spring Security

Cấu hình Spring Security theo mô hình stateless:

```
http
    .csrf(c -> c.disable())
    .cors(c -> c.configurationSource(corsSource()))
    .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(a -> a
        .requestMatchers("/api/auth/**", "/swagger-ui/**",
            "/v3/api-docs/**").permitAll()
        .anyRequest().authenticated())
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
```

JwtUtils.generate() tạo token với các claims: sub (userId), email, role và expiration 24 giờ. JwtAuthFilter đọc header Authorization, xác thực token và thiết lập Authentication context cho mỗi request.

6.5 Xử lý lỗi toàn cục

GlobalExceptionHandler (@RestControllerAdvice) xử lý tập trung các loại exception và trả về JSON chuẩn hóa:

- ApiException: Custom exception với HttpStatus linh hoạt (400, 401, 403, 404, 500).
- MethodArgumentNotValidException: Trả về danh sách field errors khi validation thất bại.
- Mọi response lỗi tuân theo cấu trúc: { "error": "...", "fields": {...} }.

6.6 Tích hợp AI qua WebClient

AIClient.predict() thực hiện gọi FastAPI với cơ chế đáng tin cậy:

```
AiResponse body = aiWebClient.post()
    .uri("/predict")
    .bodyValue(new AiRequest(prompt))
    .retrieve()
    .bodyToMono(AiResponse.class)
```

```
.retryWhen(Retry.fixedDelay(2, Duration.ofMillis(500))  
  .filter(ex -> !(ex instanceof IllegalArgumentException)))  
.timeout(Duration.ofSeconds(30))  
.block();
```

Timeout kết nối và đọc/ghi được cấu hình qua ai.timeout-ms (mặc định 30,000ms).
Retry 2 lần với delay 500ms khi gặp lỗi kết nối. Latency được đo bằng
System.currentTimeMillis() trước và sau khi gọi AI.

7. THIẾT KẾ CƠ SỞ DỮ LIỆU

Cơ sở dữ liệu gồm 3 bảng chính:
User, Conversations, Messages

7.1 Chi tiết bảng users

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	Khóa chính
username	VARCHAR(64)	UNIQUE, NOT NULL	Tên đăng nhập
email	VARCHAR(128)	UNIQUE, NOT NULL	Địa chỉ email
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu mã hóa BCrypt
role	ENUM('USER','ADMIN')	NOT NULL	Vai trò người dùng
created_at	DATETIME	DEFAULT NOW()	Thời điểm tạo tài khoản

7.2 Chi tiết bảng conversations

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	Khóa chính
user_id	BIGINT	FK → users.id, ON DELETE CASCADE	Người sở hữu conversation
title	VARCHAR(255)	NOT NULL	Tiêu đề phiên dịch
created_at	DATETIME	DEFAULT NOW()	Thời điểm tạo

7.3 Chi tiết bảng messages

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
id	BIGINT	PK, AUTO_INCREMENT	Khóa chính

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
conversation_id	BIGINT	FK → conversations.id, ON DELETE CASCADE	Conversation chứa tin nhắn
sender_type	ENUM('USER','AI')	NOT NULL	Người gửi
original_text	TEXT	NOT NULL	Văn bản gốc
translated_text	TEXT	NULL	Văn bản đã dịch
latency_ms	INT	NULL	Thời gian phản hồi AI (ms)
created_at	DATETIME	DEFAULT NOW()	Thời điểm tạo

8. TÍCH HỢP AI INFERENCE

8.1 Mô hình sử dụng

Dự án sử dụng mô hình ngôn ngữ EnViT5, một mô hình Transformer được tinh chỉnh (fine-tuned) cho nhiệm vụ dịch thuật y khoa giữa tiếng Anh và tiếng Việt. Mô hình được triển khai dưới dạng REST service thông qua FastAPI.

8.2 Giao thức giao tiếp với FastAPI

Backend giao tiếp với AI service thông qua HTTP POST request tới endpoint /predict:

Tham số	Giá trị mẫu	Mô tả
Method	POST	HTTP method
Endpoint	/predict	URL AI service
Request body (EN→VI)	{ "text": "en: Please translate..." }	Prefix "en:" chỉ định hướng dịch
Request body (VI→EN)	{ "text": "vi: Vui lòng dịch..." }	Prefix "vi:" chỉ định hướng dịch
Response	{ "output": "Kết quả dịch..." }	Kết quả dịch thuật từ mô hình

8.3 Xử lý hướng dịch

Backend chuẩn hóa prompt theo hướng dịch được người dùng chọn:

```
if ("EN_VI".equalsIgnoreCase(direction)) {
    prompt = "en: " + req.getText();
} else if ("VI_EN".equalsIgnoreCase(direction)) {
    prompt = "vi: " + req.getText();
} else {
    throw ApiException.badRequest("Invalid direction");
}
```

Sau khi nhận kết quả, hàm cleanOutput() loại bỏ các tiền tố "en:" hoặc "vi:" có thể xuất hiện trong output của mô hình để đảm bảo nội dung hiển thị sạch sẽ.

8.4 Đo lường Latency

Thời gian phản hồi AI được đo chính xác tại tầng AiClient:

```
long start = System.currentTimeMillis();
// ... gọi WebClient ...
long latencyMs = System.currentTimeMillis() - start;
// Lưu latencyMs vào bảng messages
```

Giá trị latency_ms được lưu vào cơ sở dữ liệu cùng với message và hiển thị trên giao diện với biểu tượng ⚡.

8.5 Xử lý lỗi AI Service

- Kết nối thất bại sau 2 lần retry: Trả `ApiException` với thông báo "AI Server is unavailable".
- Response null hoặc rỗng: Trả `ApiException` với thông báo "Empty AI response".
- Timeout sau 30 giây: Kích hoạt `TimeoutException`, được bắt và xử lý bởi `GlobalExceptionHandler`.

9. CÁC CHỨC NĂNG CHÍNH VÀ LUỒNG HOẠT ĐỘNG

9.1 Đăng ký và Đăng nhập

Luồng đăng ký: Người dùng nhập username, email, password → Backend validate (@NotBlank, @Email, @Size) → Kiểm tra trùng lặp → BCrypt hash password → Lưu user → Trả JWT token.

Luồng đăng nhập: Người dùng nhập email + password → Backend xác thực credential → BCrypt matches() → Generate JWT → Frontend lưu token vào Zustand store.

9.2 Quản lý Conversation

- Tạo mới: POST /api/conversations tạo conversation với title "New chat"; title được cập nhật tự động theo nội dung dịch đầu tiên.
- Liệt kê: GET /api/conversations trả danh sách conversation của user, sắp xếp theo thời gian tạo mới nhất.
- Xóa: DELETE /api/conversations/{id} kiểm tra ownership (trả 403 nếu không có quyền), sau đó xóa cascade toàn bộ messages.

9.3 Dịch thuật thời gian thực

Đây là luồng nghiệp vụ trung tâm của hệ thống:

6. Frontend gửi POST /api/messages/translate với { conversationId, text, direction }.
7. Backend xác thực JWT và kiểm tra conversation thuộc user.
8. Lưu message người dùng (sender_type = USER) vào bảng messages.
9. Chuẩn hóa prompt theo direction và gọi AiClient.predict().
10. Nhận kết quả từ FastAPI, chạy cleanOutput(), tính latencyMs.
11. Lưu message AI (sender_type = AI) với translated_text và latency_ms.
12. Nếu conversation có title "New chat", cập nhật title theo 50 ký tự đầu của văn bản gốc.
13. Trả toàn bộ kết quả về frontend để hiển thị.

9.4 Danh sách API Endpoints

Method	Endpoint	Auth	Mô tả
POST	/api/auth/register	Không	Đăng ký tài khoản mới
POST	/api/auth/login	Không	Đăng nhập và nhận JWT
GET	/api/conversations	JWT	Lấy danh sách conversation
POST	/api/conversations	JWT	Tạo conversation mới
DELETE	/api/conversations/{id}	JWT	Xóa conversation theo ID
GET	/api/conversations/{id}/messages	JWT	Lấy lịch sử tin nhắn
POST	/api/messages/translate	JWT	Gửi yêu cầu dịch thuật

10. BẢO MẬT

10.1 Xác thực và Phân quyền

- JWT Stateless: Token không lưu server-side, toàn bộ thông tin xác thực trong payload.
- Header chuẩn: Authorization: Bearer <token> theo chuẩn RFC 6750.
- Auto-logout: Frontend tự động đăng xuất và xóa token khi backend trả 401 Unauthorized.
- Route protection: Spring Security bảo vệ tất cả route trừ /api/auth/** và Swagger docs.

10.2 Bảo mật mật khẩu và JWT

- BCrypt hashing: Mật khẩu được mã hóa bằng BCryptPasswordEncoder với strength factor mặc định (10 rounds).
- JWT secret: Cấu hình trong application.yml; khuyến nghị sử dụng chuỗi ngẫu nhiên độ dài ≥ 64 ký tự trong production.
- JWT expiration: Token hết hạn sau 24 giờ; cần bổ sung refresh token cho production.

10.3 Validation dữ liệu đầu vào

Backend sử dụng jakarta.validation để kiểm tra dữ liệu DTO trước khi xử lý:

- @NotBlank: Các trường bắt buộc không được rỗng.
- @Email: Định dạng email hợp lệ.
- @Size: Giới hạn độ dài username, password.
- @NotNull: Các trường không được null.

10.4 Cấu hình CORS

```
CorsConfiguration c = new CorsConfiguration();
c.setAllowedOrigins(List.of(allowedOrigins.split(", ")));
c.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
c.setAllowedHeaders(List.of("*"));
c.setAllowCredentials(true);
```

CORS được cấu hình chỉ cho phép origin cụ thể (mặc định http://localhost:5173) thông qua biến môi trường cors.allowed-origins.

11. HƯỚNG DẪN CÀI ĐẶT VÀ CHẠY DỰ ÁN

11.1 Yêu cầu hệ thống

Thành phần	Phiên bản yêu cầu	Ghi chú
Java JDK	17 trở lên	Oracle JDK hoặc OpenJDK
Maven	3.6 trở lên	Hoặc sử dụng Maven Wrapper ./mvnw
Node.js	18 trở lên	Kèm npm 9+
MySQL	8.0 trở lên	Chạy trên port 3306
FastAPI	Latest	Dịch vụ AI bên ngoài, cần chạy riêng

11.2 Bước 1: Khởi tạo Cơ sở dữ liệu

```
# Đăng nhập MySQL và tạo database
mysql -u root -p

# Chạy schema migration
mysql -u root -p < database/schema.sql
```

11.3 Bước 2: Cấu hình Backend

Mở file backend/src/main/resources/application.yml và cập nhật các thông số:

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/medtrans
    username: YOUR_DB_USERNAME
    password: YOUR_DB_PASSWORD

  jwt:
    secret: YOUR_STRONG_SECRET_KEY_MIN_64_CHARS
    expiration-ms: 86400000 # 24 giờ

  ai:
    base-url: http://localhost:8000 # URL FastAPI service
    timeout-ms: 30000

  cors:
    allowed-origins: http://localhost:5173
```

11.4 Bước 3: Chạy Backend

```
cd backend

# Chạy ứng dụng
./mvnw spring-boot:run

# Hoặc build và chạy JAR
./mvnw clean package -DskipTests
java -jar target/medtrans-*.jar

# Chạy unit tests
./mvnw test
```

Backend sẽ khởi động tại <http://localhost:8080>. Swagger UI có thể truy cập tại <http://localhost:8080/swagger-ui.html>.

11.5 Bước 4: Cấu hình và chạy Frontend

```
cd frontend

# Sao chép file cấu hình môi trường
cp .env.example .env

# Chỉnh sửa .env
VITE_API_URL=http://localhost:8080

# Cài đặt dependencies
npm install

# Chạy development server
npm run dev

# Build production
npm run build
```

Frontend development server sẽ khởi động tại <http://localhost:5173>.

12. KẾT QUẢ ĐẠT ĐƯỢC

12.1 Danh sách tính năng hoàn thành

Tính năng	Trạng thái	Ghi chú
Đăng ký & Đăng nhập JWT	Hoàn thành	Bao gồm validation đầy đủ
Quản lý Conversation (CRUD)	Hoàn thành	Với kiểm tra ownership
Lịch sử tin nhắn phân biệt User/AI	Hoàn thành	Bubble chat có màu phân biệt
Dịch thuật AI với FastAPI/EnViT5	Hoàn thành	Retry & timeout được cấu hình
Hiển thị Latency theo thời gian thực	Hoàn thành	⚡ {ms} trên mỗi message AI
Dark mode & Responsive UI	Hoàn thành	TailwindCSS dark class strategy
Axios interceptor JWT tự động	Hoàn thành	Auto-logout khi 401
Swagger API Documentation	Hoàn thành	Tại /swagger-ui.html
Auto-update Conversation title	Hoàn thành	Cập nhật từ nội dung đầu tiên
Copy kết quả dịch	Hoàn thành	Clipboard API + toast xác nhận

12.2 Hiệu năng hệ thống

Thông số hiệu năng điển hình được ghi nhận trong quá trình kiểm thử:

- Backend REST API: Thời gian phản hồi nội bộ dưới 100ms cho các thao tác CRUD thông thường.
- Dịch thuật AI: Latency tổng phụ thuộc chủ yếu vào thời gian inference của mô hình EnViT5 trên FastAPI.
- Giá trị latency_ms được lưu và hiển thị minh bạch cho người dùng trên từng tin nhắn.

12.3 Mô tả giao diện người dùng

Giao diện MedTrans được thiết kế theo layout 3 phần:

- Sidebar trái: Hiển thị danh sách conversation với tên và thời gian tạo; nút "New Chat" để tạo mới; nút Logout ở cuối.
- Header trên: Dropdown chọn hướng dịch (EN→VI / VI→EN) và toggle dark/light mode.
- Chat area giữa: Hiển thị bubble tin nhắn (màu xanh cho user, màu xám cho AI), latency indicator và nút copy.
- Input bar dưới: Textarea auto-resize, placeholder gợi ý, nút Send và phím tắt Enter.

13. HƯỚNG PHÁT TRIỂN

13.1 Tính năng nâng cao

Xây dựng một chatbot về các vấn đề y tế, thiết kế theo mô hình rag để mô hình tránh trả lời sai các vấn đề nhạy cảm như chủ đề về y tế.

Sử dụng mô hình envit5 sau khi finetune để dịch văn bản tiếng anh thành tiếng việt, nâng hiệu suất truy vấn dữ liệu tiếng việt được lưu trong chroma.

14. KẾT LUẬN

Dự án MedTrans đã xây dựng thành công một hệ thống dịch thuật y khoa trực tuyến đầy đủ chức năng, kết hợp các công nghệ hiện đại trong phát triển phần mềm. Kết quả đạt được chứng minh khả năng thiết kế và triển khai một giải pháp phần mềm hoàn chỉnh theo đúng quy trình kỹ thuật chuyên nghiệp.

Về mặt kỹ thuật, dự án đã đạt được các mục tiêu đề ra:

- Kiến trúc phân lớp rõ ràng, tách biệt hoàn toàn frontend và backend, dễ bảo trì và mở rộng.
- Tích hợp thành công AI Inference bên ngoài thông qua giao thức HTTP chuẩn với cơ chế retry và timeout đáng tin cậy.
- Xử lý authentication/authorization theo mô hình JWT stateless hiện đại và an toàn.
- Quản lý đầy đủ vòng đời hội thoại và lịch sử tin nhắn với cơ sở dữ liệu quan hệ chuẩn hóa.
- Giao diện người dùng trực quan, responsive, hỗ trợ dark mode và các shortcut tiện lợi.

Hệ thống hiện tại đã đặt nền tảng vững chắc cho việc phát triển tiếp theo. Với các cải tiến về bảo mật dữ liệu y tế, hạ tầng container hóa và nâng cao chất lượng mô hình AI, MedTrans có tiềm năng trở thành công cụ hỗ trợ thiết thực cho cộng đồng y tế Việt Nam trong việc tiếp cận và xử lý tài liệu y khoa đa ngữ.